Systems Specifications

Interoperability, Syntax, and Semantics in Distributed Systems

Martin Benning

University College London

ENGF0034 – Design and Professional Skills

Lecture Overview

Objectives

To understand the role of specifications in ensuring software interoperability, specifically within the context of the Pac-Man coursework. We will examine:

- The distinction between Requirement, Formal, and Interoperability specifications.
- The core components of a specification: **Semantics** (Meaning) and **Syntax** (Encoding).
- Design choices in distributed systems (Dead Reckoning, UDP vs TCP).
- Trade-offs between textual and binary encodings.

Lecture Overview

The Purpose of Specification

Why Write a Specification?

Writing a specification is time-consuming; we must justify the effort.

1. Requirement Specification

- Defines *what* the software must do before it is written.
- A collaboration between the developers and the clients.
- Used to verify if the final product meets the need.

2. Formal Specification

- A mathematical expression of behaviour.
- Useful for pinning down complex logic or safety-critical behaviour.

Why Write a Specification?

Writing a specification is time-consuming; we must justify the effort.

1. Requirement Specification

- Defines *what* the software must do before it is written.
- A collaboration between the developers and the clients.
- Used to verify if the final product meets the need.

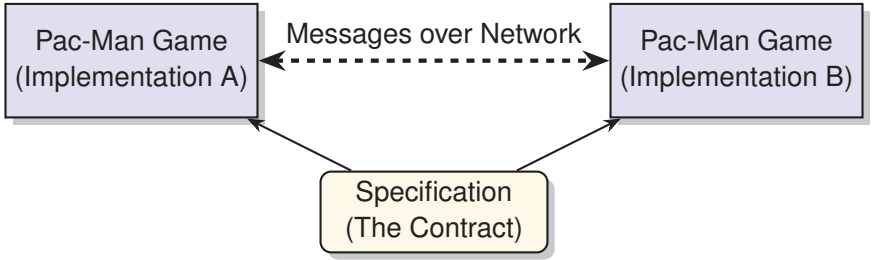
2. Formal Specification

- A mathematical expression of behaviour.
- Useful for pinning down complex logic or safety-critical behaviour.

Our Focus: Interoperability

For this course, we focus on specifications intended to allow different pieces of software (possibly written in different languages or on different OSs) to talk to each other.

The Interoperability Contract



The Goal

You do not need to know how the other side is implemented. You only need to know enough to send messages that will be interpreted correctly.

Semantics: Meaning and Behaviour

Defining Semantics

Semantics concerns the *meaning* of the messages exchanged.

Three Key Semantic Questions

1. **Timing:** When do we actually send messages? When is it important to notify the other side?
2. **Information:** What data is in the message, and what does it signify? (e.g., Does ”Ghost 3” refer to a specific entity?)
3. **Action:** What should the recipient *do* upon receipt? How should they update their state?

Case Study: Pac-Man Position Updates

How do we keep two remote Pac-Man games synchronised?

Design Choice A: Simple Updates

- Send an update every frame (30-60Hz).
- Receiver simply maps Pac-Man to the coordinate received.
- **Pros:** Simple to implement.
- **Cons:** ”Chatty” (high bandwidth); susceptible to network lag, leading to ”glitchy” motion.

Case Study: Pac-Man Position Updates

Design Choice A: Simple Updates

- Send an update every frame (30-60Hz).
- Receiver simply maps Pac-Man to the coordinate received.
- **Pros:** Simple to implement.
- **Cons:** ”Chatty” (high bandwidth); susceptible to network lag, leading to ”glitchy” motion.

Design Choice B: Dead Reckoning

- Send updates less frequently.
- Include **Position**, **Direction**, and **Speed**.
- **Receiver Action:** In between updates, animate (predict) Pac-Man moving along vector.
- **Result:** Smoother motion, even if the network is imperfect.

Network Reality: Reliability vs. Latency

The network is not perfect. We must choose a transport protocol that dictates part of our semantics.

UDP (User Datagram Protocol)

- Sends message immediately.
- **Unreliable:** Packets may be lost and never arrive.
- **Use Case:** Position updates (losing one is okay).
- **Problem:** Critical events (e.g., ”Pac-Man Died”) must not be lost. You must implement your own retry logic.

TCP (Transmission Control Protocol)

- **Reliable:** Retransmits lost data automatically.
- **Latency:** Loss causes delay while waiting for retransmission.
- **Problem:** Late arrival of position data can cause ”rubber-banding” (jumping backwards/forwards).

Syntax: Encoding the Data

Defining Syntax

Once we know *what* to send (Semantics), we must define *how* to encode it (Syntax).

The Encoding Spectrum

- **Textual Encoding:** Human-readable, uses a defined grammar (e.g., JSON, XML, Custom Text).
- **Binary Encoding:** Machine-efficient, packs bits/bytes directly (e.g., TCP Headers, Structs).

Trade-off

Text: Flexible, readable, but inefficient (parsing overhead, larger size).
Binary: Efficient, compact, but rigid and hard for humans to debug.

Textual Encoding Example: URLs

URLs (RFC 3986) are a classic textual specification.

Grammar (ABNF)

They are defined using a formal grammar called Augmented Backus-Naur Form (ABNF).

Structure

```
scheme ":" hier-part [ "?" query ] [ "#" fragment ]
```

- Allows hierarchy: `http://authority/path`
- Restricted character set: Only US-ASCII (alphanumerics, digit, hyphen, dot, underscore, tilde).

Textual Encoding Example: URLs

URLs (RFC 3986) are a classic textual specification.

Grammar (ABNF)

They are defined using a formal grammar called Augmented Backus-Naur Form (ABNF).

Structure

```
scheme ":" hier-part [ "?" query ] [ "#" fragment ]
```

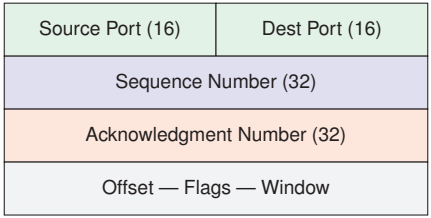
- Allows hierarchy: `http://authority/path`
- Restricted character set: Only US-ASCII (alphanumerics, digit, hyphen, dot, underscore, tilde).

Internationalization

Since URLs are ASCII, Unicode (e.g., Chinese characters) must be mapped to ASCII

Binary Encoding Example: TCP Headers

TCP (RFC 793) prioritizes efficiency over flexibility.



Characteristics

- **Fixed Layout:** Fields are always in the same place. Easy for hardware/software to parse.
- **Compact:** Headers are small to minimise overhead.
- **Limited Flexibility:** Can add ”Options” (Type-Length-Value) at the end, but the core is

Binary Encoding Challenges

When sending binary data (e.g., integers), you must be precise.

Endianness (Byte Order)

Computers store multi-byte integers differently.

- **Big Endian:** Most significant byte first (Network Standard).
- **Little Endian:** Least significant byte first (x86 Standard).

The Specification Requirement

Your specification must explicitly state the byte order. *Historical Note:* RFC 793 forgot to specify this, but ”Big Endian” became the de-facto standard. You should not rely on luck; specify it!

Scope and Context

Establishing Boundaries

A good specification defines its scope: what is included and, crucially, what is **excluded**.

The Lower Bound: Infrastructure

Do **not** specify how the internet works (IP) or how transport protocols (TCP/UDP) function.

- These are existing standards (RFCs).
- They are already implemented in the OS.

The Upper Bound: Game Logic

Do **not** specify the internal rules of Pac-Man.

- Assume the game logic (ghost behaviour, scoring) is a ”fixed point”.

The Task

Summary

Summary

Key Takeaways

- Specifications for this course are contracts for **interoperability**.
- **Semantics** define the ”what” and ”when”:
  - Consider update frequency vs. smoothness (Dead Reckoning).
  - Choose the right protocol (UDP vs TCP) based on loss tolerance vs. latency sensitivity.
- **Syntax** defines the ”how”:
  - *Textual:* Readable, flexible (Grammars/Parsers).
  - *Binary:* Efficient, rigid. Must specify **Endianness** and types.
- Define the **Scope**: Don’t reinvent TCP/IP or the Game Engine; specify the glue between them.